It’s an older code but it checks out – Michael Waterman

michaelwaterman.nl/2022/12/15/its-an-older-code-but-it-checks-out

Michael Waterman December 15, 2022

The Windows operating systems from its humble beginning up to the point where it is today has gone through tremendous chances to battle all the emerging security threads. Among those are the most rudimentary forms of disruption up to the very sophisticated state sponsored attacks which can lead to the destabilization of society. As the saying goes in the industry, it’s always a rat race.

The Cyber criminals find a new vulnerability to misuse and it’s up for us as defenders to plug that hole and make sure it can never happen again. But it’s not always the latest and the greatest that’s being used to compromise and extort an organization. Reading trough, the latest Microsoft Digital Defense Report 2022 I was amazed by the number of incidents where credential theft was at the heart of the attack. Cyber criminals would compromise a system, dump the credentials, initiate a lateral movement, dump additional credentials and do a whack a mole on your systems. Perhaps we need to have a refresher on keeping credentials safe. When I was still working for Microsoft, we released the “Windows 10 Credential Theft Mitigation Guide”. Although it’s been a couple of years, the conceptual architecture that’s laid out in that document is still very valid even in the light of Cloud usage but especially with hybrid or on-prem systems. In the next couple of posts, I will be focusing on exactly these mitigating technologies that will keep your credentials safe.

Introduction

One of the most common forms of attack include the theft and misusage of credentials. Not just the logon credentials of end-users but also from privileged users such as admins or

domain admins. Using a technology named “Pass the Hash” attackers can use tools such as “Mimikatz” or the Windows Credential Editor to obtain and misuse credentials to eventually steal data or worse destroy an entire company’s infrastructure. Although there have been updates for both Windows 7 and Windows 8.1 trying to mitigate these forms of attack, it’s more than just adding additional protective capabilities. The problem has always been that the credentials are stored on the operating system itself. If, somehow, your machine is compromised, it’s no longer your machine and the attacker can pretty much do anything with it. That anything usually includes getting the credentials from the local administrator account, helpdesk employees and even domain admins. Although guidance exists in the PTH whitepapers, most companies seem to be ignoring the fact that they are vulnerable to these forms of attack. I must admit, some of the proposed solution have a huge impact on the way organizations manages or supports their computers.

Windows NT History

Back in 1993 Windows NT 3.1 was released to market and the biggest customer of Microsoft at that time, (and probably still) is the United States Department of Defense. And although no official documentation exists that state that they demanded NT to be built in a certain way, I can only image them having said something along the lines of, “If you don’t build it a certain way, we can’t buy it”. The lead architect at the time was Dave Cutler, who joined the company in 1988 and was made responsible for the creation of, what would later be known as Windows NT.

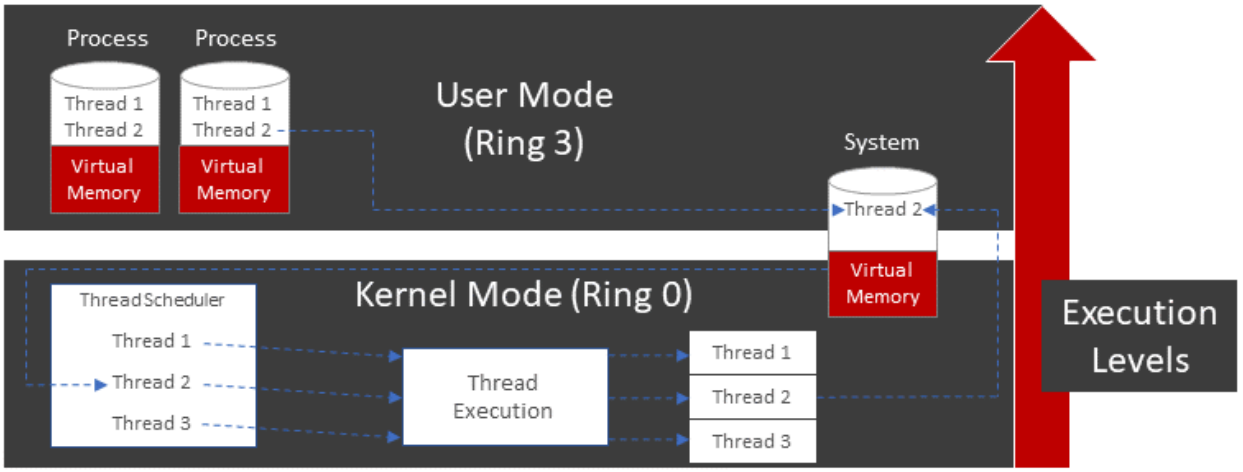
During the development of Windows NT, the team had to decide on which processor architecture the system would run and how it would use that processor. Although I’m in no means an expert when it comes to silicon usage, I do get the basic concepts of processor execution levels and the security behind it. That brings me to the concept of processor rings.

A processor is divided into rings and each ring represents the ability to handle instructions. As you can imagine, instructions will, to a certain degree, have a security impact. Craving them up into rings controls execution of processor instructions. imagine you as a user being able to read secure data such as operating system secrets, you could compromise an entire system instantly. Divided instructions on your processor prevent that from happening.

During development of Windows NT, it was decided that, because of the DoD “compliancy”, only processor ring zero and three would be used. We know these rings today as user and kernel mode. So, in essence we have two modes in Windows, one where all the user mode code execution is done and security is very strict, and one where kernel mode code runs and security is relaxed because of performance requirements.

Architecture

Let's first talk about the Windows Architecture that has been around from 1993 up until today, yes you're reading this correctly, the same architecture is still in place today. To be honest, not a lot has been changed over the years, but as one of my colleagues used to say, everything has been altered a bit. From a high level perspective, the image below represents the topics that we will be talking about.



Kernel Mode

Code that is executed in kernel mode has total and unrestricted access to the underlaying hardware, meaning that code running in the kernel can also read every physical memory block and use every instruction available on the processor. Hence that only trustworthy processes should be running here. Traditionally kernel mode has the highest privilege or execution level. In Windows the binary that represents the kernel is "ntoskrnl.exe".

User Mode

The major difference between Kernel and User mode is that code executed in user mode can't directly interact with the hardware. Every execution is required to talk to the system code (APIs) and that in turn interacts with the hardware on behalf of the user mode process. Memory allocation from User and Kernel mode is one of those topics that's relevant for this blog post.

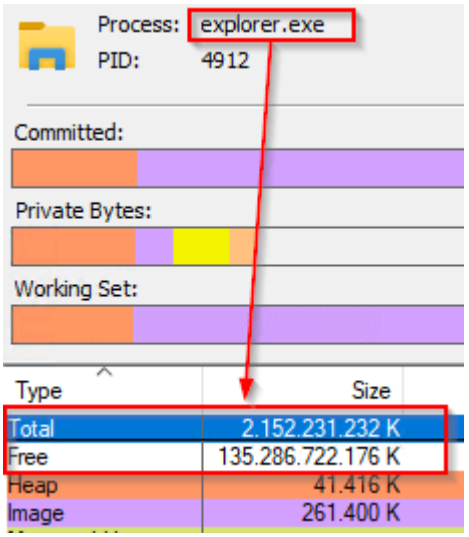
Windows Memory Management

An application on the disk is what we call an (binary) image, once this image is executed, it in turn is put in memory and identified as a single process or multiple dependent processes. Looking at the Windows operating system specifically, a process is what we see in Task Manager. Each of the processes that we see contains of a number of items, but for the sake of this post, we focus on two items. Virtual Memory and threads.

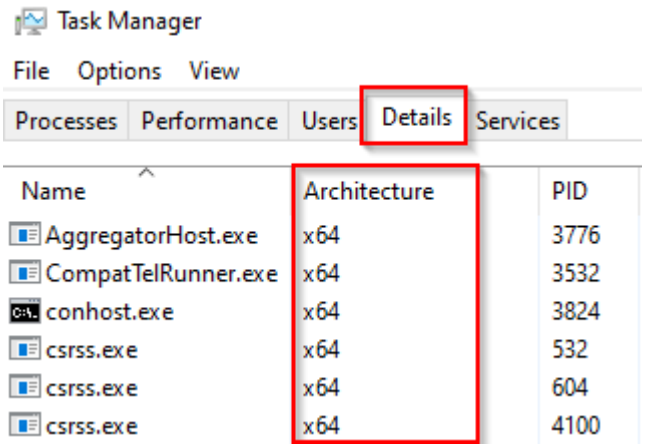
Virtual Memory

In the beginning of time application themselves would be responsible for memory management, which in many cases would lead to application and the Operating System crashing because of inadequate memory usage. Another challenge was memory usage itself; some application would consume all the memory available, leading to system instability. To handle this, the concept of virtual memory was introduced. Virtual memory in essence is a virtual representation of memory to an application running in user mode. Virtual memory does not exist in kernel mode.

On Windows 11 virtual memory has a total size of around 136 TB per process. Let that sink in for a moment. There's so much more memory available per process than the physical available memory on an average system. For any application to run out of virtual memory these days is very unlikely. To easily see what any process is consuming use the Sysinternals tool, VMMap.



Please note that the process needs to be 64Bit to address this amount of virtual memory, 32Bit applications (available in SysWow64 directory) can only address 4Gig max. To identify the process type, look in Task Manager at the details tab, specifically the "Architecture" column.



Any process created in user mode uses virtual memory and starts writing its first byte at the virtual address 0. As said, this is translated into physical memory eventually. One thing I want you to remember from all of this is that the translation is referred to as "*First Level Address Translation*". It will be explained further on why that's important.

Threads

Last on the list are threads. Every process that's started on the Windows Operating System consists of threads. You can think of a thread as being the unit that is able to be computational processed on the CPU. Processes are not directly handled, only threads. In essence see it as carving up a process and sending small bits of it towards the CPU. As a small sidestep, just for fun. There's something named a fiber, that is a thread that's managed by a process for optimized memory management. Not a lot of application use it though because it can get complex very quickly. Most notable is SQL server that optimizes a system this way for its database transactions.

Context Switching

Now that we know what User and Kernel mode are, how process with virtual memory and threads work, lets look at the last concept that I would like to address. Context switching.

Context switching is the ability of a thread to move between User and Kernel mode. But I told you in the beginning this was impossible because of how the processor architecture works, there are exceptions, however. Think about your video card for example. Although a lot of the processing nowadays is in user mode, there are still drivers involved that live in the kernel and they need to be able to communicate with threads and processes in user mode. On Windows this is made possible by the process we all know as "System". Hate to break it to you, but System is a process much like notepad, with the exception that the System process lives in between worlds. Once a user mode thread needs to process its code in kernel mode, it hands the thread over to the system process that acts as a gatekeeper between worlds. Much like Heimdall in the Marvel movies. The thread is assigned a new attribute, allowing it passage to the world of the kernel. After processing the thread travels back to the world of user mode where the System process again assigns a new attribute allowing it to travel back.

Credentials theft and reuse

Now to the problem itself. Many of the attacks that we see nowadays are based on the reusage of stolen credentials. These credentials can come from anywhere and putting up multifactor authentication can make a huge difference. But suppose an attacker is already within the compounds of your infrastructure? Without the proper protection they could very likely dump credentials from memory and reuse those to eventually reach your Domain

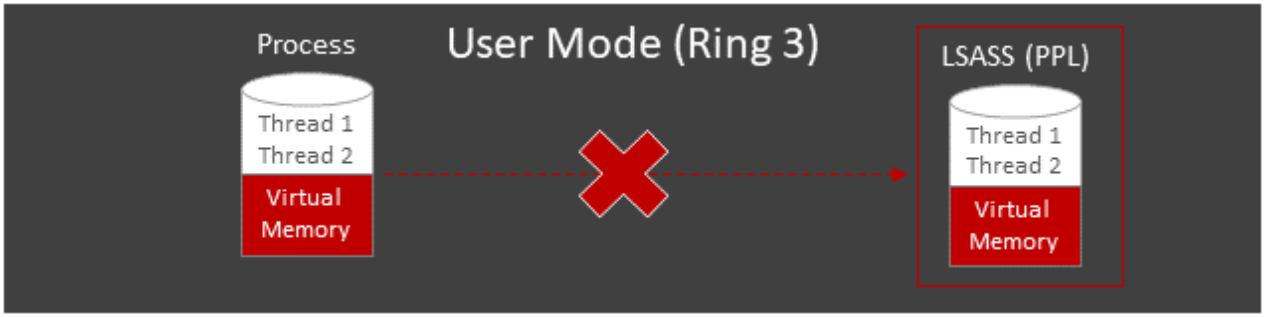
Controllers and steal your data. This problem isn't new, it's been around for many years now and is still very valid today.

LSASS

On Windows the Local Security Authority Subsystem is responsible for safeguarding the keys to the kingdom. These include NTLM hashes, Kerberos tickets and anything that has to do with authentication and authorization, including enforcing the security policies. A part of this subsystem is responsible for maintaining the credentials that are generated during the session. That process is the Local Security Authority Server Service (LSASS) . Although it's deemed as a protected process, over the years tools like MimiKatz are still able to extract and reuse credentials that are stored in the memory location of the LSASS process. Even something as trivial as a process or full memory dump would contain the credentials and Mimikatz would be able to extract the credentials and eventually inject it into a new process, thereby reusing them. I'll do a blog about that fun stuff at a later stage.

Protected Process

To combat the bad guys from stealing credentials, the introduction of Windows Server 2012 R2 and 8.1 a feature was made available called LSA (Local Security Authority) Protection Light. This allows LSASS to run as a very controlled protected process.



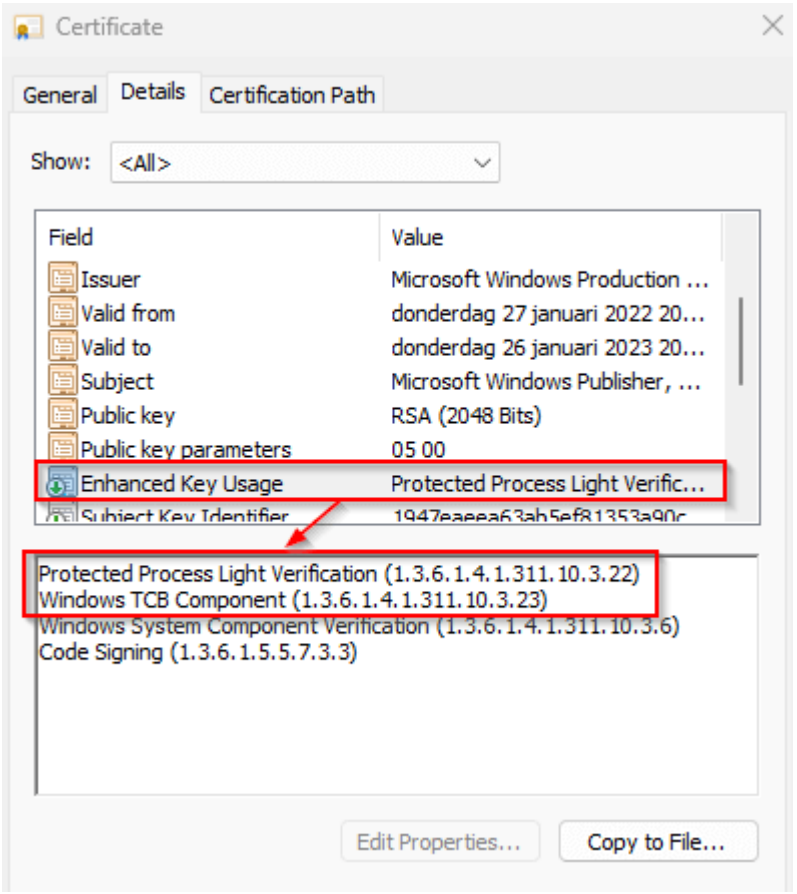
The protection that was used was derived from the Windows Vista/2008 days and allows regular processes to access protected processes in a limited way. Back in the day the primary purpose was for supporting DRM, not having your best interest at heart. Protection for LSASS is a PPL implementation, meaning a Protect Process Light and allowed for an access level protection on top of the protected process. This means that within the APIs that were defined for a protected process, PPL allows for even more granular access control. Access control is defined by a signer level and the type.

Protection Process Level	Type	Signer	Access Level
PS_PROTECTED_SYSTEM	Protected (2)	WinSystem (7)	0x72
PS_PROTECTED_WINTCB	Protected (2)	WinTcb (6)	0x62
PS_PROTECTED_WINTCB_LIGHT	Protected Light (1)	WinTcb (6)	0x61
PS_PROTECTED_WINDOWS	Protected (2)	Windows (5)	0x52
PS_PROTECTED_WINDOWS_LIGHT	Protected Light (1)	Windows (5)	0x51
PS_PROTECTED_LSA_LIGHT	Protected Light (1)	LSA (4)	0x41
PS_PROTECTED_ANTIMALWARE_LIGHT	Protected Light (1)	Anti-malware (3)	0x31
PS_PROTECTED_AUTHENTICODE	Protected (2)	Authenticode (1)	0x21
PS_PROTECTED_AUTHENTICODE_LIGHT	Protected Light (1)	Authenticode (1)	0x11
S_PROTECTED_NONE	None	None	0x0

As you can see in the access level column, there is a certain degree of hierarchy. Meaning that a process at the top can access all underlaying processes but not the other way around. Although there are a couple of rules to it:

- A protected process can access another protected process or a protected process light when the accessing protected process operates on the same or higher access level as the process it’s trying to access.
- A protected process light can access another protected process light when the accessing protected process light operates on the same or higher access level as the process it’s trying to access.
- A protected process light cannot access a protected process regardless of level.

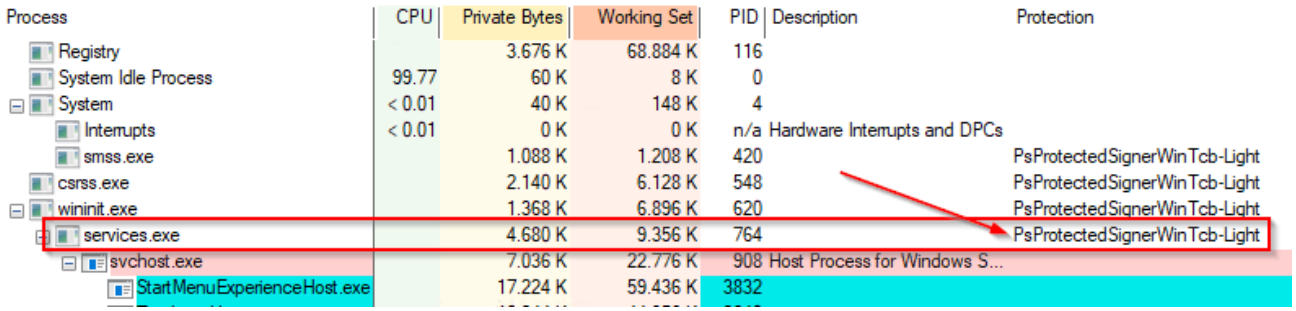
In case you're wondering how you can determine the access level, well there's the hard and easy way. The hard way would be to look at the signing properties of the binary file. The enhanced key usage has two entries that define the access level.



In the example above, signing is displayed for services.exe. It has the following two values:

- Protected Process Light Verification (1.3.6.1.4.1.311.10.3.22)
- Windows TCB Component (1.3.6.1.4.1.311.10.3.23)

Looking at the table above this would create a Protected Process Light (1) with the signer level WinTcb (6). Giving it an access level of 0x61. The easier way would be to use Sysinternals Process Explorer. Enable the column "Protection" and look at the result.



To summarize, here are the access levels and their respected usage.

Signer name	Level	Description
PsProtectedSignerWinSystem	7	System and minimal processes.
PsProtectedSignerWinTcb	6	Critical Windows components.
PsProtectedSignerWindows	5	Important Windows components handling sensitive data.
PsProtectedSignerLsa	4	Lsass.exe (if confi gured to run protected).
PsProtectedSignerAntimalware	3	Anti-malware services and processes, including third party. PROCESS_TERMINATE is denied.
PsProtectedSignerCodeGen	2	NGEN (.NET native code generation).
PsProtectedSignerAuthenticode	1	Hosting DRM content or loading user-mode fonts.
PsProtectedSignerNone	0	Not valid (no protection).

Enabling PPL for LSASS

Turning the protective feature on is really easy. Just use this registery key, reboot the machine and that's it.

```
HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL=1 (DWORD)
```

Windows Registry

Note! Just remember one thing. If you're using secure boot and enable RunAsPPL, an attribute is written in the UEFI framework and removing the registry key will not remove the protection. You would have to physically remove it by resetting secure boot.

Note! More info on RunAsPPL can be located here: Configuring Additional LSA Protection | Microsoft Learn

No dice

You could think, "Okay, so problem solved, move along". Not really, remember I talked about protected processes. And where do processes live? Right, in user mode and anything in kernel mode has unrestricted access to memory in user mode. That's exactly what the developer of Mimikatz did. He developed a driver (mimidrv.sys) that could be injected into kernel memory and again have full access to LSASS. Just give it a try yourself with the following commands:

```
!+
!processprotect /process:lsass.exe /remove
privilege::debug
sekurlsa::logonpasswords
```

BAT (Batchfile)
For this to be successful the driver needs to be in the same folder as Mimikatz. Fortunately, all AV systems will detect this driver, but there's nothing stopping you from creating your own version and bypassing protections.

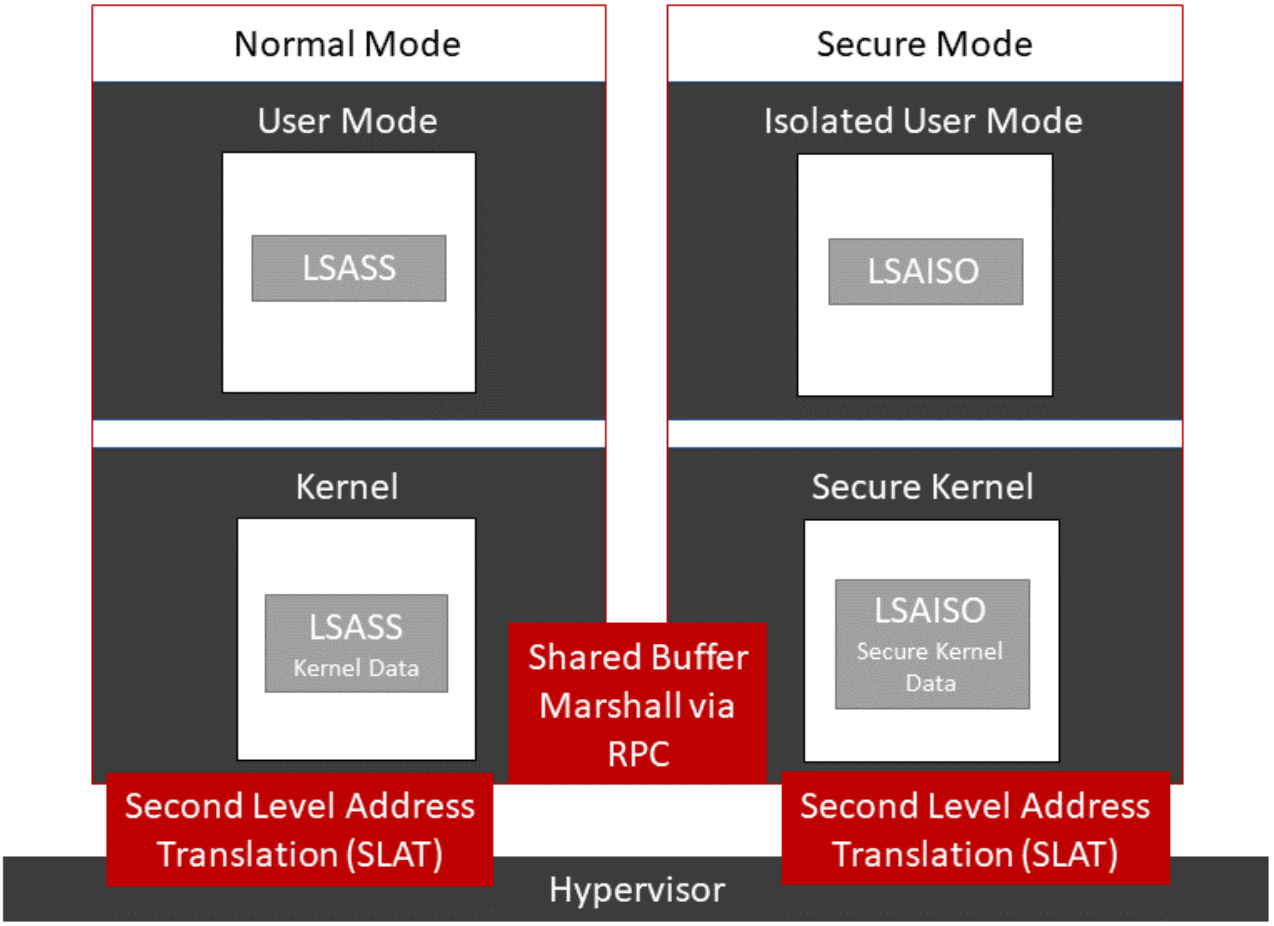
Windows Defender Credential Guard

Ever since the introduction of Windows 10, enterprises had the option to safeguard the user credentials, blocking the current versions of the well know credential theft tools. Now with the release of Windows 11 22H2, this functionality is turned on by default.

Windows 10 Enterprise, Windows Server 2016+ and Windows 11 (Pro+) make it possible to enable a new way of storing user credentials in a shielded environment, separated from the main operating system. Only authorized processes are allowed to gain access using a secured RPC connection. This secured environment is based on a virtualization layer. One could think that a complete instance of Windows would be running there, but that's not the case. Only the required binaries and of course the credentials are stored there. An additional layer of security is present in the form of binary signing. All files that operate in that specific virtual secure environment have been signed so they can't be tempered with. I can imagine that this sounds a bit like magic, so let's take a step back and explain what's going on.

Two worlds apart

Remember what I said when explaining kernel mode? It has unrestricted access to the hardware, meaning memory and all. Moving the credentials from user mode to kernel mode wouldn't be a solution because as demonstrated by Mimikatz, a driver would gain access to that memory space as well. That's where Credential Guard comes in. In essence a brand-new kernel next to the ntoskrnl.exe has been build that separates user mode and kernel mode even further and in essence brings us four processor rings. That new kernel, "securekernel.exe" is much smaller and acts as a custodian for secrets we call trustlets. Among these trustlets is the LSASS process.



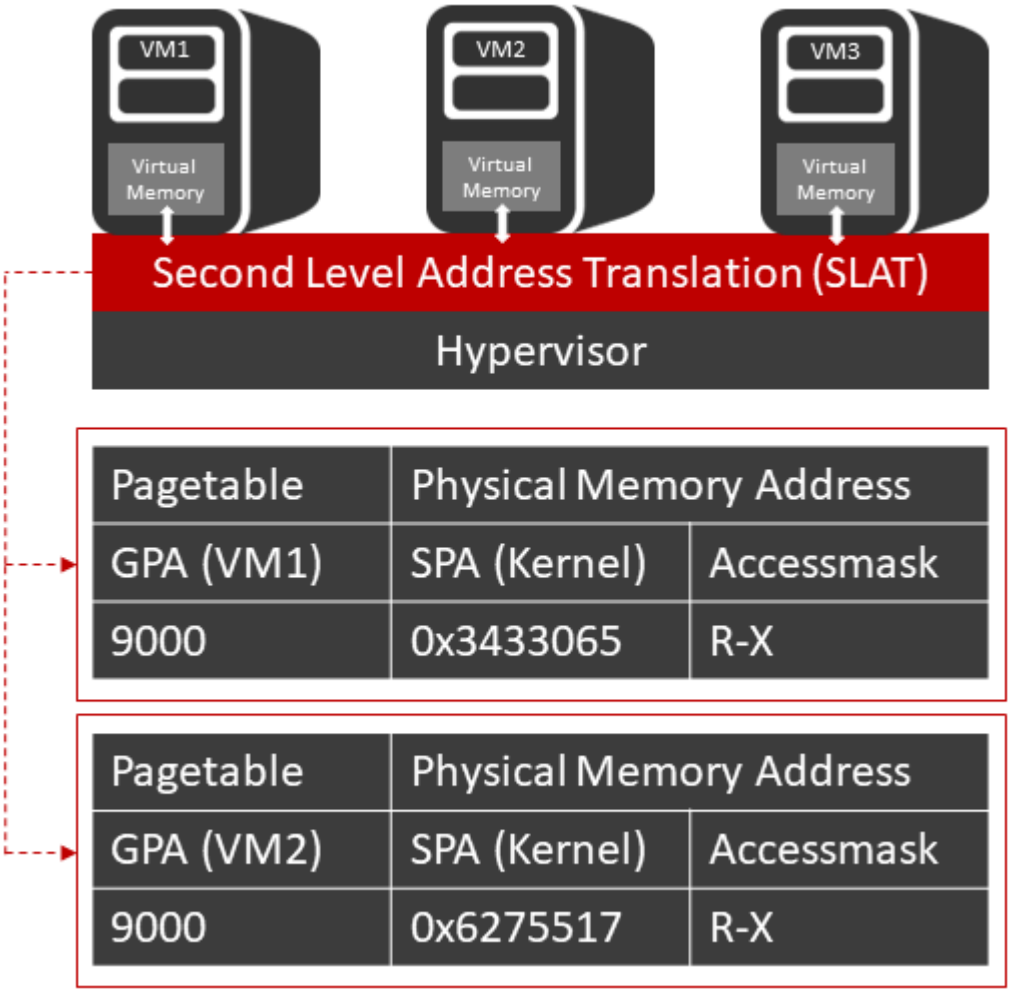
As you can see in the image, there are four worlds. User Mode, Isolated User Mode, Kernel mode and the Secured kernel mode. But to understand how this all works together we must first look at a technology that's been around for a long time, virtual machines on top of a hypervisor.

Hypervisor and virtual machines

Virtualization has been around for a long time and brings lots of enhancements for the optimized usage of hardware. To be able to do that, virtualization splits hardware resources like CPU, physical disk usage and memory. It's the latter part of the technology that's a core component of Credential guard.

Much like processes that I described in the previous paragraphs, virtual machines aren't that different. VM's are assigned their own virtual hardware that the operating system believes is physical. Virtual memory for example is a certain amount of the physical available memory but the operating system inside the VM only sees the amount that has been assigned to it. If you've been reading this entire blog this must sound familiar, it's very similar to processes that also have virtual memory assigned. Once virtual memory from the process is translated into physical memory it's called "First Level Address Translation", guess what, we name the translation of Virtual Machine memory to physical memory, "**S**econd **L**evel **A**ddress **T**ranslation", or SLAT. It's a memory optimization technique that

physically separates memory at the hardware level. What if we could use such a technology to separate credentials? Luckily, we can do that.



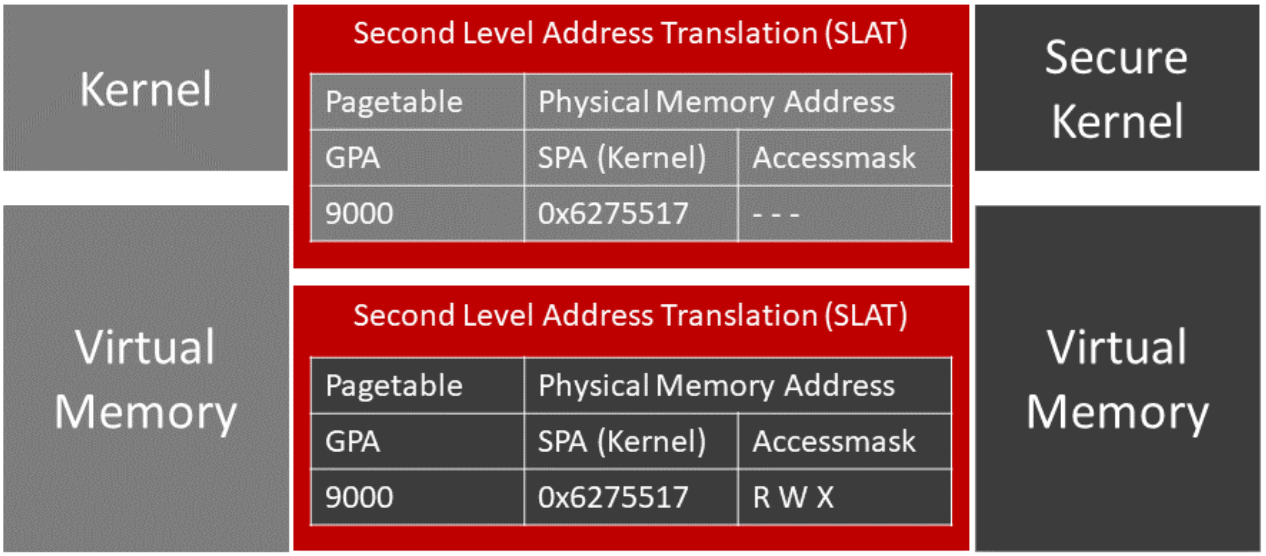
SLAT from the hardware perspective uses Intel VT or AMD-V for its functions and before it was introduced the memory translation would be done by the hypervisor itself. To understand how the separation works you need to understand the memory components involved.

- **SPA**, the System Physical Address.
- **GPA**, Guest Physical Address.
- **GVA**, Guest Virtual Address.

Looking at the SPA component, this translates to the physical available memory on your computer and is controlled by the hypervisor layer. GPA, contrary to popular belief is also physical memory, but committed to the virtual machine or child partition as dictated in Hyper-V. GVA is defined as the virtual memory that is associated with the virtual machine. In this setup memory has to be translated twice, GVA -> GPA -> SPA. This double address translation is now moved from the hypervisor to the hardware (SLAT) where it can be processed faster and more efficient.

Virtual Trust Levels

The primary component within SLAT that's responsible for this optimization is called "Extended Page Tables" (EPT) and that's where the magic happens. EPT's are equipped with a Virtual Trust Levels (VTL) that can separate memory from being accessed by a lower level VTL or by the root partition, a.k.a. the Operating System kernel.



As displayed in the image above you can actually see that there are two SLATs defined that use an EPT/VTL that in turn use an ACL (RWX) to block memory access. As a result the kernel (NTOSKRNL) cannot access the memory addresses that are used by the Secure Kernel.

> *What's actually been created here is a separation of memory access by means of an Access control list (ACL) controlled by virtualization technology.*

Meaning that although conceptually a kernel should be able to read all memory, it's now blocked in certain areas. The mentioned ACL supports 3 levels:

- Block or read (R): In essence this is Credential guard that allows or blocks memory reads.
- Block or read Execute: Device guard that allows or denies code execution.
- Block or write: block or allow the modification of executable pages shared with VTL1.

Secure Kernel

Now we know how memory is isolated from the secure kernel, but what is this new secure kernel exactly?

When you only look at the size comparison between the two kernels, 11 MB for the regular NTOSKRNL, versus just 1 megabyte for the secure kernel, there is a lot that's not programmed into the secure kernel itself. From an efficiency point of view, why would you

have duplicate code in two kernels anyway? It doesn't make a lot of sense. The secure kernel provides just the code for the secure operations and functions as a proxy when it wants to use a function that's available in the ntoskrnl. Take writing pages to disk as an example. Suppose a memory page available only to the secure kernel needs to be paged out to disk, it would be proxied to the ntoskrnl and written to disk. But wait, that means that the kernel again would have access to memory it isn't supposed to have. Luckily that's covered by the implementation of cryptocode within the secure kernel. All memory that's proxied is encrypted by the secure kernel before being transferred to the regular kernel.

Isolated user mode and Trustlets.

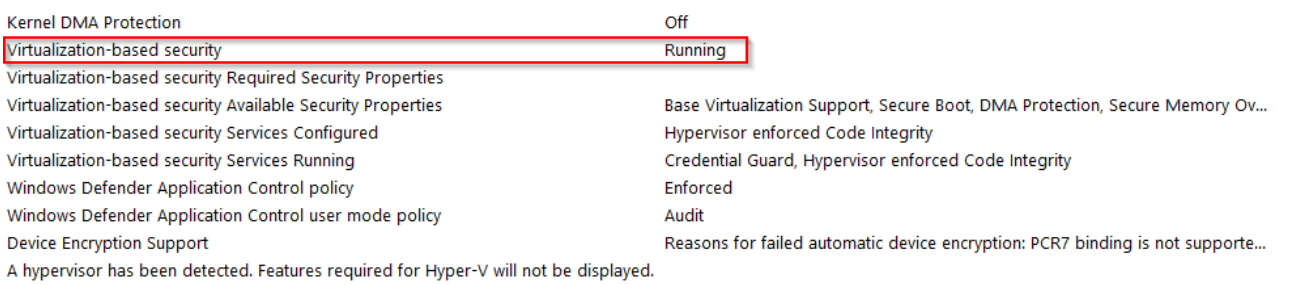
Last but not least, we need to talk about Isolated user mode (IUM), or better I should say IUM processes, AKA trustlets. Same thing, just different names. In essence these are the user mode processes protected by the credential guard technology described in this blog. All memory that's being used by these processes is protected by the same technology and can't be reached by even the regular kernel, thus keeping secrets very secure. Among the trustlets available in IUM are:

- Credential Guard (LSAISO as process name)
- Biometrics information
- Virtual TPM
- Kernel Mode Code Integrity
- Windows Hello for business
- Windows Defender Application Guard

To sum it all up, data from IUM will need to flow through the secure kernel where it will pass the shared buffer through the kernel and, if needed, end up in user mode. Obviously, this will include the challenge / response process result, not the secrets themselves.

Identifying Credential Guard

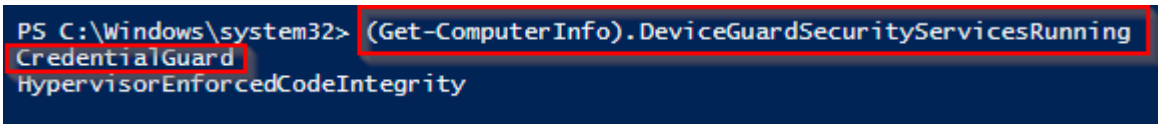
Identifying if credential guard is enabled or not is very easy, you can choose between using the GUI or PowerShell. If choosing the GUI, simply run "*msinfo32.exe*" and scroll down to the system summary page. You should search for "*Virtualization-based security*" as depicted in the image below.



Using PowerShell, you just need to run a single line of code:

```
(Get-ComputerInfo).DeviceGuardSecurityServicesRunning
```

PowerShell



If the output contains “CredentialGuard” you should be good.

Hopefully you’ve found this information useful and now understand how something that’s relatively simple to enable has a huge impact on protecting your secrets.

Reference

I’ve used the following links to make this blog post:

Windows NT history

Oral History of Dave Cutler Part 1 – YouTube

Oral History of Dave Cutler Part 2 – YouTube

Windows Defender Credential Guard – Microsoft Learn

How Windows Defender Credential Guard works | Microsoft Learn

Manage Windows Defender Credential Guard (Windows) | Microsoft Learn

Isolated User Mode in Windows 10 with Dave Probert | Microsoft Learn

More on Processes and Features in Windows 10 Isolated User Mode with Dave Probert | Microsoft Learn

Isolated User Mode Processes and Features in Windows 10 with Logan Gabriel | Microsoft Learn

Sami Laiho – The Master on this topic, from his teachings I learned the most.

Learn about Windows 10 Secure Kernel – Events | Microsoft Learn